

SIGEE | parecer da entrega

Pietro e Diego • Revisão em 12/09/2026 • Prazo interno: 21/09 • Entrega: 28/09

O trabalho cobre os quatro itens técnicos da etapa, com uma ressalva no aceite versionado. A entrega acadêmica completa ainda depende de corrigir esse caso, atualizar as evidências e confirmar o TCC consolidado e a validação do orientador.

Exigência de 28/09	Resultado da análise
Login: autenticação, autorização e criptografia	Atendido no recorte técnico. Login/logout nativos, permissões no servidor, senha com hash e páginas públicas com HTTPS/HSTS. Fluxos autenticados validados localmente.
Auditoria/log de acessos e ações	Atendido no recorte básico. Eventos persistentes, consulta restrita e integração às operações atuais. Os limites de cobertura estão explícitos na página 4.
Informações sobre LGPD: Termo e Política	Páginas atendidas; aceite com ressalva. Documentos públicos, dados/finalidades descritos e confirmação registrada. Há um defeito quando a versão muda com formulário aberto.
API externa: projeto lógico + documentação	Atendido como projeto. BrasilAPI de feriados: endpoint, fluxo, dados, timeout e contingência documentados. A chamada executável segue fora desta etapa.
Desenvolvimento documentado e validado com orientador	Parcialmente comprovado. Há documentação técnica no repositório. O TCC consolidado e a evidência de validação do orientador não foram disponibilizados nesta revisão.

Versões e autoria das frentes

Pietro - autenticação/perfis: branch `login-auth-segurança`, commit `def22f9`, [PR #6](#). **Diego - auditoria/privacidade:** branch `auditoria`, commit `bf84b69`, [PR #7](#). Divisão conforme o plano e o resumo recebido; ambos os PRs foram abertos pela conta PietroNozella.

Os dois PRs já estão integrados. A versão conjunta analisada é `93a0cdd`, cujo conteúdo é idêntico ao da branch `auditoria`. A main local permanecia em `503fa06`, três commits atrás. Os testes usaram cópias isoladas, sem checkout, merge ou alteração de código do projeto.

Resultados e ajustes necessários

Evidências reproduzidas nesta análise; resultados históricos identificados separadamente.

Verificação	Resultado
Branch de autenticação	57/57 testes aprovados (10,060 s).
Auditoria + autenticação integradas	102/102 testes aprovados (15,033 s). A suíte já inclui a autenticação; os números não devem ser somados como testes distintos.
Verificação de esquema e configuração	Ambas as versões: system check sem problemas; makemigrations --check --dry-run sem alterações pendentes.
Seis testes extras desta revisão	Cinco passaram; um falhou. Passaram CSRF no login, auditoria de CSRF negado, expiração da sessão e rollback de usuário/equipamento quando a auditoria falha.
Fluxo real no navegador local	Administrador: login → aceite → cadastro de Professor → evento no log → logout. Professor criado: login → aceite → inventário; acesso à auditoria retorna 403. Tela de auditoria conferida em desktop e 390 px.
Ambiente publicado, acesso anônimo	Login, Termos e Política: HTTP 200 por HTTPS/HSTS. Inventário e aceite levam ao login. Status Vercel do commit integrado: success.

P2 - aceite pode registrar uma versão não apresentada

Reprodução: abrir / aceite/ na versão 1.0; mudar TERMOS_USO_VERSAO para 2.0; enviar o formulário antigo. O servidor responde 302 e grava 2.0. O serviço lê as versões vigentes no POST, sem conferir as exibidas no GET. Isso compromete a evidência de qual documento foi confirmado. Fontes: legal/services.py:22-28; legal/views.py:50-53; template de aceite:32-46.

Correção mínima proposta: enviar as duas versões apresentadas no formulário e compará-las com as atuais no servidor. Se alguma mudou ou estiver ausente, não gravar; exibir as novas versões e pedir nova confirmação. Adicionar o teste GET 1.0 → mudança para 2.0 → POST antigo. Não é necessário criar outra arquitetura.

P3 - evidência de deploy desatualizada

README.md:164 ainda informa necessidade de corrigir um 404 observado em 10/09; docs/evidencias-seguranca.md:72 preserva esse resultado. Em 12/09 o login respondeu 200. Manter a evidência histórica com sua data e acrescentar a verificação atual, incluindo o aceite no fluxo de autenticação.

Limites da verificação: Python 3.12.14, Django 5.2.17 e SQLite isolado. Sem ensaio autenticado em produção, validação de PostgreSQL ou medição de cobertura percentual. A contagem de testes não comprova os 50% previstos em RNF-06. O status Vercel comprova deploy, não uma revisão humana ou aprovação acadêmica.

Pietro | autenticação e perfis

RF-01, RNF-01, RS-01, RS-04, RS-05 e RN-18 • Trechos reais da versão integrada.

Autenticar e encerrar a sessão

config/urls.py : 23-30

```
path(
    'login/',
    auth_views.LoginView.as_view(
        template_name='registration/login.html',
    ),
    name='login',
),
path('logout/', auth_views.LogoutView.as_view(), name='logout'),
```

A rota utiliza LoginView e LogoutView do Django. O formulário valida as credenciais, inicia a sessão e preserva um destino seguro em next. O logout aceita POST; os templates enviam token CSRF. Credenciais inválidas recebem uma mensagem neutra.

Aplicar autorização no servidor

inventario/views.py : 20-22

```
@login_required
@permission_required("inventario.view Equipamento", raise_exception=True)
def equipamento_lista(request):
```

login_required encaminha o visitante ao login. permission_required exige a permissão de consulta e devolve 403 para a conta autenticada sem acesso. Cadastro, CSV e exclusão usam suas permissões correspondentes. A ocultação dos botões é somente uma ajuda visual.

Em usuarios/permissoes.py, cadastro de contas e consulta de auditoria exigem exclusivamente o grupo Administrador e a permissão da ação, excluindo superuser técnico. No inventário, valem as permissões nativas, inclusive suas regras de superuser.

Criar contas sem conceder privilégios técnicos

usuarios/forms.py : 60-72

```
def save(self, commit=True):
    user = super().save(commit=False)
    user.is_staff = False
    user.is_superuser = False

    if not commit:
        return user

    with transaction.atomic():
        user.save()
        user.groups.set([self.cleaned_data["perfil"]])

    return user
```

CadastroUsuarioForm estende UserCreationForm: a senha passa pelos validadores e pelo hash do Django. A transação salva a conta e um único grupo. is_staff e is_superuser permanecem falsos. Nome, sobrenome, e-mail, username, perfil e senha inicial são obrigatórios; não existe cadastro público.

Configuração e segurança: configurar_perfis cria/reaplica os três grupos após migrate. Administrador gerencia o inventário; Operador e Professor consultam. O hash protege a senha armazenada; HTTPS protege o tráfego. A versão integrada ativa redirecionamento HTTPS e cookies Secure quando DEBUG=False. A sessão usa o padrão de 14 dias do Django, sem prazo próprio de inatividade.

Diego | auditoria integrada

RS-09 e RS-11 • Model, serviço, sinais e chamadas explícitas nas operações.

Guardar eventos pequenos e rastreáveis

auditoria/models.py : 18-22

```
acao = models.CharField(max_length=100)
resultado = models.CharField(max_length=20, choices=Resultado.choices)
data_hora = models.DateTimeField(auto_now_add=True, db_index=True)
entidade = models.CharField(max_length=100, blank=True)
entidade_id = models.CharField(max_length=100, blank=True)
```

O evento também tem usuario opcional com PROTECT, impedindo excluir contas referenciadas. Banco e serviço validam os campos. O ID textual preserva a referência após excluir um equipamento.

Capturar autenticação sem copiar credenciais

auditoria/signals.py : 18-23

```
@receiver(user_login_failed, dispatch_uid="auditoria_login_falhou")
def registrar_login_falhou(sender, credentials, request, **kwargs):
    registrar_evento(
        acao=AcaoAuditoria.LOGIN_FALHOU,
        resultado=RegistroAuditoria.Resultado.FALHA,
    )
```

O sinal gera falha anônima, sem guardar nome ou senha tentados. Outros sinais registram login e logout; o middleware registra respostas 403 de contas autenticadas.

Salvar a operação junto com a auditoria

usuarios/views.py : 24-32

```
with transaction.atomic():
    usuario_cadastrado = form.save()
    registrar_evento(
        usuario=request.user,
        acao=AcaoAuditoria.USUARIO_CADASTRADO,
        resultado=RegistroAuditoria.Resultado.SUCESSO,
        entidade=usuario_cadastrado._meta.label,
        entidade_id=usuario_cadastrado.pk,
    )
```

Conta e evento pertencem à mesma transação: falha no log desfaz o cadastro. O inventário aplica esse princípio a cadastro, importação e exclusão/inativação. Os testes extras confirmaram o rollback de conta e equipamento.

Consulta restrita e limites assumidos

A tela /auditoria/ permite ao Administrador funcional filtrar por usuário, ação, resultado e datas, com 20 itens por página. É somente leitura; o Django Admin também bloqueia incluir, editar e excluir logs.

Cobertura: autenticação, 403 autenticado, busca filtrada, download CSV, cadastro de equipamento, importação, exclusão/inativação, cadastro de usuário e aceite. A importação gera um evento por lote, sem IDs individuais; abrir o inventário sem filtros não gera evento.

Limites: não copia senhas, hashes, tokens, formulários/CSV, IP ou buscas. O bloqueio na interface não impede alteração por acesso direto ao banco. Assinatura dos eventos e módulos futuros estão fora desta etapa.

Diego | privacidade e aceite

RS-10 e RS-12 • Páginas públicas, controle de versão e ligação à auditoria.

Informar antes da autenticação

Termos de Uso e Política de Privacidade estão públicos e ligados ao login e à navegação interna. Descrevem contas, sessões, auditoria, movimentações, finalidades, conservação, responsabilidades e contato configurável. O usuário aceita os Termos e confirma a ciência da Política; a documentação distingue esse registro de uma autorização genérica para tratar dados.

Bloquear a área interna até a confirmação

[legal/middleware.py](#) : 25-34

```
if (
    getattr(usuario, "is_authenticated", False)
    and not self._rota_isenta(request.path_info)
    and not usuario_possui_aceite_vigente(usuario)
):
    parametros = ""
    if request.method in {"GET", "HEAD"}:
        parametros = f"?{urlencode({'next': request.get_full_path()})}"
    return redirect(f"{reverse('legal:aceite_documentos')}{parametros}")
```

Depois de identificar a conta, o middleware verifica o aceite vigente. Sem ele, redireciona para /aceite/ e preserva next em GET/HEAD. Login, logout, documentos, aceite, arquivos estáticos e área técnica /admin/ são exceções. As duas confirmações são obrigatórias.

Evitar duplicidade e preservar o histórico

Em [legal/models.py](#), UniqueConstraint permite apenas um aceite por usuário e combinação de versões. A relação com a conta usa PROTECT; o Django Admin bloqueia alterações pela interface. Uma mudança de versão exige outro registro.

[legal/services.py](#) : 22-28

```
def registrar_aceite_vigente(usuario):
    versao_termos, versao_privacidade = versoes_atuais()
    return AceiteDocumentosLegais.objects.get_or_create(
        usuario=usuario,
        versao_termos=versao_termos,
        versao_privacidade=versao_privacidade,
    )
```

get_or_create evita recriar um aceite existente. A view grava o aceite e DOCUMENTOS_LEGAIS_ACEITOS na mesma transação. Este é também o ponto do defeito descrito na página 2: usa a versão atual do servidor, sem verificar qual foi apresentada no formulário.

Leitura acadêmica do resultado

O requisito de disponibilizar informações de privacidade está implementado. O aceite versionado acrescenta uma evidência útil, mas deve receber o ajuste identificado. O resumo do Diego é coerente com o código e sua contagem de 102 testes foi reproduzida; as instruções de publicar branch/abrir PR no resumo já ficaram históricas porque o PR #7 foi integrado.

A revisão técnica não comprova, sozinha, conformidade integral com a LGPD. O próprio documento do projeto reserva à instituição a definição de base legal, responsabilidades e retenção. A ANPD também trata a proteção de dados como combinação de medidas técnicas e administrativas [5]. Para a etapa acadêmica, apresentar com precisão o que foi implementado e o que depende da implantação institucional.

BrasilAPI e fechamento da etapa

RF-09 projetado • Entrega completa prevista para 21/09, uma semana antes de 28/09.

Projeto lógico entregue por Pietro

Elemento	Definição documentada
Endpoint e entrada	GET https://brasilapi.com.br/api/feriados/v1/{ano} ; somente o ano derivado do período de reserva. Uma consulta por ano envolvido.
Resposta utilizada	Lista JSON com date, name e type; seleção dos feriados entre início e fim, inclusive. Saída interna: datas, nomes e indicação de consulta completa/indisponível.
Falhas e tempo limite	Timeout proposto de 3 segundos por ano; tratar rede/TLS, HTTP inesperado, JSON inválido e campos ausentes. Avisar e permitir continuar a reserva.
Cenários previstos	Sem feriado, vários feriados, passagem de ano e indisponibilidade. Nenhuma resposta externa decide disponibilidade do equipamento ou conflito de reservas.

O documento **docs/projeto-logico-brasilapi.md** distingue projeto de implementação. Não há cliente HTTP nem reserva funcional; isso corresponde ao recorte combinado da imagem, que pede projeto lógico e documentação. Os cenários da API são futuros, não resultados de testes executados. A arquitetura geral e seu diagrama preexistiam aos dois PRs e foram mantidos.

Cinco ações para encerrar com evidências

Quando / responsável	Resultado necessário
Até 14/09 - Diego	Corrigir o envio de formulário antigo após mudança de versão e incluir o teste de regressão. Pietro revisa a correção.
Até 16/09 - Pietro + Diego	Atualizar o resultado do deploy e o fluxo com aceite; entregar ao documento acadêmico as seções, trechos e evidências de cada frente.
Até 17/09 - ambos	Conferir migrations, configurar _perfis e o fluxo autenticado no ambiente de apresentação, usando contas de demonstração. Registrar o commit e o resultado.
17-18/09 - ambos	Apresentar os quatro itens ao orientador e registrar retorno. Não equiparar merge ou deploy à validação acadêmica.
Até 21/09 - ambos	Incorporar os ajustes aceitos, conferir PDF final do TCC e congelar a versão de demonstração. Manter 22-28/09 como margem para imprevistos.

Adequação ao oitavo semestre: a solução mantém monólito Django, templates, ORM e recursos nativos de segurança, sem dependências novas nesses PRs. A separação em usuarios, auditoria e legal é compreensível e tem responsabilidades concretas. Cada autor deve conseguir explicar a permissão, a transação, o evento gerado e o teste que demonstra sua parte. Não há motivo identificado para um refactor amplo.

Fontes da revisão

[1] [PR #6](#) e [2] [PR #7](#): metadados, branches, histórico e estado de integração. [3] [Código e documentação em 93a0cdd](#); trechos das páginas 3-5 ligados às linhas originais. [4] [SIGEE_resumo_branch_auditoria.pdf](#), 3 páginas, e imagem da exigência de 28/09 fornecidos pelo usuário. [5] [ANPD: Guia de segurança da informação, versão 1.0, seções 2-3](#). Consulta e testes próprios em 12/09/2026.